Recommendation Pipeline - Design

- Dataset: 100,000 public MovieLens interactions from the ashraq/movielens_ratings mirror, plus its movie catalog.

- The mirror has no authoritative event timestamps; timestamps are deterministically generated (NumPy seed 42).

- Interactions retained: 100,000; quarantined: 0.

- Chronological holdout: 80,000 older training interactions, 20,000 newer test interactions.

- Stages: ingest, validate/quarantine, chronological preparation, feature snapshots, two models, and ranking evaluation.

- Features are stored as Parquet entity snapshots; the interaction matrix is stored as sparse NPZ.

- Models: interaction-count popularity baseline and personalized truncated-SVD collaborative filtering.

- GitHub repository: https://github.com/QuickLearnerQ/DM4ML

Recommendation Pipeline - Results

model	RMSE	MAE	Precision@10	Recall@10	NDCG@10	Coverage	Diversity
popularity	1.0134	0.7617	0.0273	0.227	0.1181	0.0018	0.8183
truncated_svd	1.0262	0.7848	0.0038	0.0279	0.0148	0.026	0.7631

- Ranking metrics use K=10 and held-out ratings of at least four stars.

- Ranking uses a deterministic sample of up to 1,000 relevant users.

- Coverage is the recommended share of the item catalog; diversity is pairwise genre Jaccard distance.

- The full comparison table is available in outputs/model_comparison.csv.

Recommendation Pipeline - Limitations

- A time-based split better approximates future interactions than a random split.

- Invalid rows are retained in data/errors with rejection reasons.

- JSONL tracking stores model parameters, metrics, and artifact paths without a tracking server.

- The Python scheduler can execute this workflow once or at a configured interval; an optional daily Airflow DAG runs the same workflow.

- MovieLens cold-start users/items, popularity bias, and offline metrics limit generalization.

- Mirror timestamps are synthetic when the source mirror does not provide event times; temporal evaluation does not represent real event chronology for this input.

Execution evidence: ingestion, validation, preparation, and feature management



Parquet features | interaction-count popularity + truncated SVD | ranking and rating metrics

1. Data ingestion

data/ml-latest-small/*.csv

```
Ratings source: 100,000 interactions
Movies: 7,312 | Links: 7,312
Columns loaded: userId, movieId, rating, timestamp
Source preview read from the ingested CSV
```

2. Validation and quarantine

outputs/validation_report.json

```
Status: passed
Valid: 100,000
Quarantined: 0
Referential integrity: {'movies': True, 'links': True}
```

3. Preparation

data/processed/train.csv, test.csv, user_item_matrix.npz

```
Train rows: 80,000 | Test rows: 20,000
Chronological: True
Last train timestamp: 1646884202
First test timestamp: 1646884248
```

4. Feature management

data/features/*.parquet and docs/feature_catalog.csv

```
users: 23,188 rows, 7 columns
items: 6,600 rows, 6 columns
interactions: 80,000 rows, 6 columns
Parquet snapshots read from the feature store
```

Visual evidence compiled from the successful run's generated files and metrics.

Execution evidence: model training, evaluation, tracking, and orchestration

5. Model training

models/*.pkl

```
popularity.pkl: present
truncated_svd.pkl: present
Artifacts checked on disk after model training
```

6. Evaluation

outputs/model_comparison.csv

```
popularity: RMSE 1.0134, MAE 0.7617, P@10 0.0273, R@10 0.2270, NDCG@10
0.1181
truncated_svd: RMSE 1.0262, MAE 0.7848, P@10 0.0038, R@10 0.0279, NDCG@10
0.0148
Values read from this run's model comparison
```

7. Experiment tracking and reporting

outputs/experiment_tracking.jsonl and reports/final_report.pdf

```
Runs recorded: 32 total
popularity: run 20261111T051413.799764Z
truncated_svd: run 20261111T051422.817563Z
Model metrics and artifact paths are recorded per run
This PDF includes the stage evidence pages
```

8. Orchestration

src/scheduler.py and dags/recommendation_pipeline.py

```
Pipeline status: passed
Python scheduler: one-shot or repeating interval
Interval option: --interval-hours
Airflow schedule: @daily
DAG configuration evidence (not an Airflow execution log)
```